

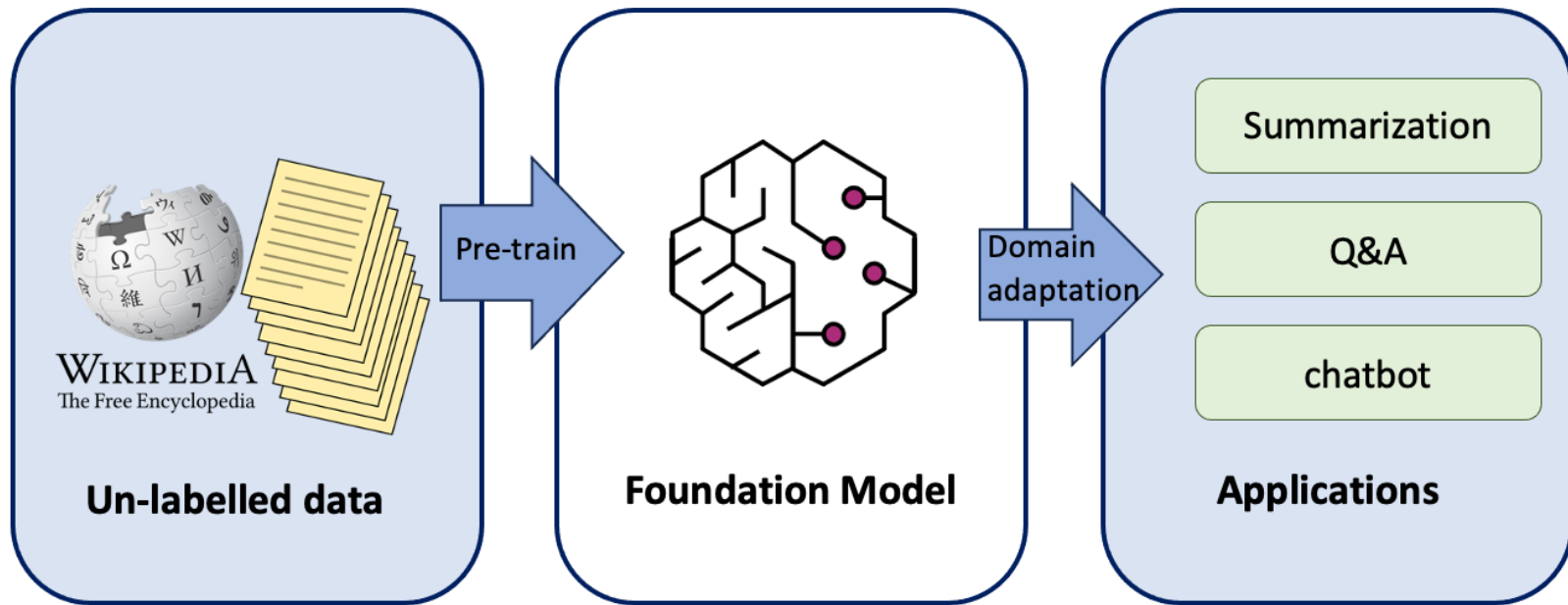


University
of Exeter

COMM117

Large Language Models and Application

Previously



Previously

Data preprocessing = transforming raw text into **clean, usable** training data

Includes:

- Cleaning
- Normalisation
- Filtering
- Deduplication
- Privacy reduction
- Quality control

Previously

- Bias
- Harmful content
- Privacy leakage
- ...

Tokenization

“Tokenization is the process of **creating a digital representation** of a real thing.

Tokenization can also be used to protect sensitive data or to efficiently process large amounts of data.”



Introduction Today

- What tokenization is, and why it matters
- Subword tokenization: BPE, WordPiece, Unigram / SentencePiece

Introduction Today

How many tokens is this sentence?

“Tokenization is important for LLMs!”

What is tokenization?

Tokenization = mapping raw text → a sequence of discrete symbols (tokens)

Text: " *Tokenization is important for LLMs !*"

Word Tokens: [*Tokenization* , is, important, for, LLMs!]

Word Tokens (punctuation split):

[*Tokenization* , is, important, for, LLMs, !]

- Tokens are the **units** used by the **model**: embeddings, attention, and prediction

Vocabulary in Tokenization

- Vocabulary (V): the fixed set of tokens your tokenizer can output
- Each token has a unique integer ID:
token $\leftrightarrow$ id where $\text{id} \in \{0, \dots, |V|-1\}$

We split each document to several tokens and map each token to an **integer** ID. An example is as follows:

Text: “I am a machine learning researcher”

Tokens: [I, am, a, machine, learning, researcher]

Token IDs: 25 7 100 2 10 1000

The vocabulary assigns each token an integer ID

Why tokenization?

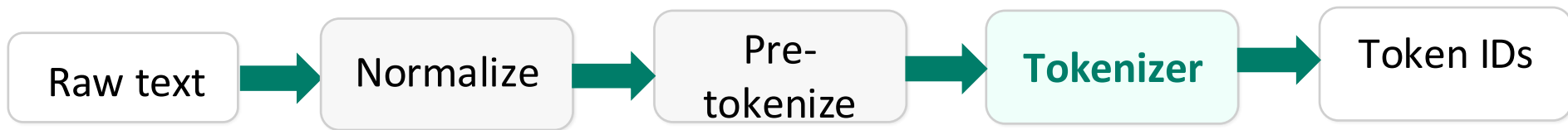
Tokenization = mapping raw text → a sequence of discrete symbols (tokens)

- **Structure raw text:** Breaking text into tokens makes it **easier to analyse and process.**
- **Preserve meaning:** **Tokens retain context and meaning** essential for tasks like sentiment analysis, translation, or summarization.
- **Standardize input:** It ensures **all inputs follow the same numerical format for the model.**

*The primary goal of tokenization is to **represent text in a manner that's meaningful for machines** without losing its context.*

Where tokenization fits?

A simplified pipeline



A tokenizer is the “text → numbers” converter for an LLM.

- Split text into tokens (often subword pieces, not whole words)
- Map each token to an ID (an integer in the model’s vocabulary)
- Reconstruct (approximately) from IDs back to text (decode)

Tokenization Type

Text: *" Tokenization is important for LLMs !"*

Word Tokens: [*Tokenization* , is, important, for, LLMs!]

Word Tokens (punctuation split):

[*Tokenization* , is, important, for, LLMs, !]

Word tokenization. This method breaks text down into **individual words**.

It's the most **common approach** and is particularly effective for languages with clear word boundaries like English.

Tokenization Type

```
text = "The quick brown fox jumps over the lazy dog"  
tokenized_text = text.split()  
print(tokenized_text)
```

```
['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']
```

Pros:

- Easy to implement.
- Retains meaningful units (words).

Cons

- **Large vocabularies:** The English language has over 500,000 words, leading to high storage requirements.
- **No semantic similarity:** Words like “run” and “running” are treated as entirely different tokens.
- **Unknown tokens:** Unrecognized words are replaced with special tokens like [UNK], leading to information loss.

Tokenization Type

Text: " *LLMs* !"

Cha Tokens: [" *L*", " *L*", " *M*", " *s*", " *!*"]

Character tokenization. the text is segmented into individual characters. Beneficial for languages that **lack clear word boundaries**, robust to **unseen words**.
But too long (compute-heavy)

Tokenization Type

```
from typing import List
def char_tokenize(text: str, keep_whitespace: bool = True) -> List[str]:
    if keep_whitespace:
        return list(text)
    return [ch for ch in text if not ch.isspace()]
```

Pros:

- **Smaller vocabularies:** Works with only a limited set of characters.
- **Minimal unknown tokens:** Rare words can still be represented since all words are built from characters.

Cons

- **Higher computational load:** A single word can result in multiple tokens.
- **Less semantic meaning:** Individual characters lack standalone meaning, except in languages like Chinese where each character conveys significant information.

Tokenization Type

Text: " Chatbots!"

Subword Tokens: [" Chat", " bots"]

Text : " #OPENAIResearch2026"

Subword : ["#", "Open", "AI", "Research", "2026"]

Subword tokenization (BPE/WordPiece/Unigram). **balance** between word and character tokenization— larger than character but smaller than a full word.

Beneficial for useful for languages that **form meaning by combining smaller units** or when dealing with **out-of-vocabulary words** in NLP tasks.

Tokenization Type

```
import re
from collections import OrderedDict, defaultdict
def preprocess_text(text):
    text = text.lower() # Convert to lowercase and handle punctuation
    # Split on whitespace and punctuation, keeping punctuation as separate tokens
    tokens = re.findall(r'\w+ | [\^\w\s]', text)
    return tokens
```

Pros:

- **Efficient vocabularies**
- **Semantic preservation**
- **Low unknown tokens**

Algorithms

- **Byte Pair Encoding (BPE):** Used in GPT-2.
- **WordPiece:** Used in BERT.
- **SentencePiece:** Used in multilingual models.

Tokenization Type

This is the idea behind **GPT-2's byte-level BPE** and similar tokenizers.

Text: " Cafe"

Byte Tokens: [63 61 66 C3 A9](UTF-8)

Text: 😊

Byte Tokens: [F0 9F 99 82] (UTF-8)

Byte tokenization. Any text string is encoded as **bytes** (values 0–255). It can represent **any input** because every possible text can be expressed as bytes.

- Encode text to **UTF-8 bytes**

However, low interpretability and token count can increase

Tokenization Type-Byte

```
from collections import Counter
```

```
# Step 1: Input vocabulary (word -> frequency)
```

```
vocab = {  
    'l o w': 5,  
    'l o w e r': 2,  
    'n e w e s t': 6,  
    'w i d e s t': 3  
}
```

```
# Step 2: Find symbol pairs
```

```
def get_pairs(vocab):  
    pairs = Counter()  
    for word, freq in vocab.items():  
        symbols = word.split()  
        for i in range(len(symbols)-1):  
            pairs[(symbols[i], symbols[i+1])] += freq  
    return pairs
```

```
# Step 3: Merge most frequent pair
```

```
def merge_vocab(vocab, pair):  
    new_vocab = {}  
    old = ''.join(pair)  
    new = ".join(pair)  
    for word, freq in vocab.items():  
        new_word = word.replace(old, new)  
        new_vocab[new_word] = freq  
    return new_vocab
```

```
# Step 4: Run BPE
```

```
num_merges = 5  
for _ in range(num_merges):  
    pairs = get_pairs(vocab)  
    if not pairs: break  
    most_common = pairs.most_common(1)[0][0]  
    vocab = merge_vocab(vocab, most_common)  
    print(f"Merged: {most_common} -> {".join(most_common)}")
```

Tokenization Type-Byte

Pros:

- Flexible vocabulary
- Handles unknown words
- Efficient representation
- Language-independent

Cons:

- Dependent on training data
- Not dynamic: Once trained, the BPE vocabulary doesn't learn new patterns unless retrained.
- Inconsistent splits: The same word might be split differently depending on context.
- No understanding

Tokenization Type

Character vs word vs subword vs byte

Unit	Pros	Cons	Typical use
Character	No OOV; robust	Very long sequences	Small models; some multilingual
Word	Readable; short seq.	OOV + huge vocab	Classic NLP; not common in LLMs
Subword	Good trade-off	Tokenizer quirks	Most modern LLMs
Byte	Covers any text	More tokens in some cases	Byte-level BPE family

Tokenization Type-Real world

- **Subword tokenization** is essential in transformer models like GPT, BERT and T5.
- GPT-2 uses Byte-Pair Encoding (BPE) on bytes, allowing it to process any Unicode text.
- BERT uses WordPiece, which selects subword units based on how likely they are to appear.
- **Vocabulary size is compact** (30k–50k tokens), making it efficient for memory and computation.
- **It replaces huge word lists** (with millions of entries) while still handling a wide variety of words.



Tiktokenizer [tiktoken app](#)

gpt2

Large Language models and Application

Token count

5

Large Language models and Application

21968, 15417, 4981, 290, 15678



token ID

☐ Show whitespace

Why subwords?

Goal: represent any text with a **manageable vocabulary**

- Word-level vocabularies explode (millions of words)
- Character-level sequences are too long for Transformers
- Subwords reuse frequent pieces (e.g., “un”, “##able”, “ing”)
- Open vocabulary: rare/new words become compositions

Byte Pair Encoding (BPE):

Overview

- Start with a **base alphabet** (characters or bytes)
- Repeatedly **merge** the most frequent adjacent pair
- Each merge **adds a new token** to the vocabulary
- After enough merges → common words/subwords become single tokens

Key idea: learn “pieces” directly from **data frequency**

Byte Pair Encoding (BPE):

- Start with a **base alphabet** (characters or bytes)
 1. Suppose we have two words in the dataset : Low and lowest
 2. The Vocabulary: {l, o, w, e, s, t, </w>}
 3. **Tokenization at Character Level:** low → ["l", "o", "w", "</w>"], lowest → ["l", "o", "w", "e", "s", "t", "</w>"]

Byte Pair Encoding (BPE):

low → ["l", "o", "w", "</w>"], lowest → ["l", "o", "w", "e", "s", "t", "</w>"]

- Repeatedly **merge** the most frequent adjacent pair

1. Counting Pair Frequencies

Identify the most frequent adjacent pairs of tokens:

- Pair l o appears **twice**: once in "low" and once in "lowest".
- Pair o w appears **twice**: once in "low" and once in "lowest".
- Pair w </w> appears **once**: in "low".
- Pair e s appears **once**: in "lowest".
- Pair s t appears **once**: in "lowest".

2. Merge the Most Frequent Pair

l o and o w → Suppose choose o w first

low → ["l", "ow", "</w>"], lowest → ["l", "ow", "e", "s", "t", "</w>"]

Byte Pair Encoding (BPE):

low → ["l", "ow", "</w>"], lowest → ["l", "ow", "e", "s", "t", "</w>"]

- Each merge **adds a new token** to the vocabulary

Vocabulary: {l, ow, e, s, t, </w>}

1. Recount Pair Frequencies

After the first merge:

- Pair **l ow** appears twice.
- Pair **e s** appears once.
- Pair **s t** appears once.



Merge **l ow**

low → ["low", "</w>"], lowest → ["low", "e", "s", "t", "</w>"]

Vocabulary: {**low**, e, s, t, </w>}

Byte Pair Encoding (BPE):

low → ["low", "</w>"], lowest → ["low", "e", "s", "t", "</w>"]

- After enough merges, common words/subwords become single tokens

Vocabulary: {low, e, s, t, </w>}

1. Recount Pair Frequencies

After the first merge:

- Pair e s appears once.
- Pair s t appears once.



Merge e s

low → ["low", "</w>"], lowest → ["low", "es", "t", "</w>"]



Finally

low → ["low", "</w>"], lowest → ["low", "est", "</w>"]

Vocabulary: {low, es, t, </w>}



Vocabulary: {low, est, </w>}

BPE: algorithm (high-level)

- Input: training corpus + **target** vocabulary size
- Count neighbor symbol pairs across the corpus
- Merge the most frequent pair → create a new symbol
- Repeat until you reach the desired vocab size
- Tokenize new text by applying the learned merges

GPT-2: byte-level BPE

Both byte-based (base symbols) and subword-like (merged tokens)

- Step 1: **encode text as UTF-8 bytes**
- Step 2: map bytes to a reversible symbol set
- Step 3: apply learned BPE merges to form subword tokens
- Result: no OOV, with compression of common patterns

Terminology: **byte-level BPE** is a subword tokenizer implemented over bytes.

Byte-level tokenization

Byte-level BPE starts from bytes (0–255) instead of Unicode characters

- Guarantees coverage: any string can be represented
- Avoids “unknown character” issues
- Often used in GPT-style tokenizers
- Trade-off: some languages may produce more tokens

Example idea: emojis / rare scripts still tokenize without UNK.

WordPiece

- Different with the BPE is the way in which symbol pairs are added to the vocabulary.
- WordPiece chooses a symbol pair which will **result in the largest increase in likelihood upon merging**.

instead of (BPE) → Repeatedly **merge** the most frequent adjacent pair

Whether to merge 'es'? $\frac{P(es)}{P(e)P(s)}$  High? Merge!

e and s appear together very often (large $P(es)$)

e and s are not just common everywhere independently (so $P(e)$ is low, $P(s)$ is low)

WordPiece

Whether to merge 'es'? $\frac{P(es)}{P(e)P(s)}$  High? Merge!

e and s appear together very often (large $P(es)$)

e and s are not just common everywhere independently (so $P(e)$ is low, $P(s)$ is low)

Example:

Total tokens: 10,000

e appears 1,000 times $\rightarrow P(e) = 0.1$

s appears 1,000 times $\rightarrow P(s) = 0.1$

es appears 3,00 times $\rightarrow P(es) = 0.03$



$$P(e)p(s) = 0.01$$
$$P(es) = 0.03$$



$$\frac{P(es)}{P(e)P(s)} = 3$$

greater than any other symbol pair?

Merge!

WordPiece vs BPE

Both build a subword vocabulary — but the objective differs

BPE

- Merge most frequent pair
- Purely frequency-driven
- Fast, simple, effective

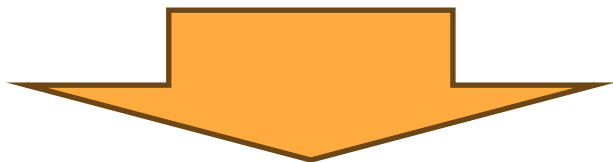
WordPiece

- Choose merges that improve a likelihood objective
- Often uses “##” to mark continue an existing word
- Popularised by BERT-family models

- In practice: you mostly care about compatibility with the target model.

Unigram

- Need: A **vocabulary** V of **learned** subword units, each with an associated **probability**.



External learned V



individual tokens

- Goal: segment text into a sequence of subword tokens that **maximizes the joint probability**.

$$P(\text{text}) = \prod_{i \sim n} P(\text{token}_i)$$

Unigram

Example: “chatbots”

Possible segmentations:

A: ["chat", "bots"] with score $P(chat)P(bots)$

B: ["chatbot", "s"] with score $P(chatbot)P(s)$

C: ["cha", "t", "bots"] with score $P(cha)P(t)P(bots)$



Largest

Pros

- It naturally prefers **fewer**, more **probable pieces**
- It is strong for **multilingual** text because it can learn appropriate pieces **without relying on whitespace rules**.
- With byte-fallback, it can always represent any text even if no good subword exists.
 - If the tokenizer cannot represent part of the text, it falls back to encoding **as raw bytes** (typically UTF-8 bytes).

Trade-offs: $|V|$ vs sequence length

Tokenizer choice is an engineering trade-off

- Large vocabulary (word-like): fewer tokens, but **larger softmax/embeddings**, more **OOV risk**
- Small vocabulary (byte/char-like): more tokens, but **universal coverage, smaller softmax**
- Subword tokenizers aim for the “sweet spot”
- Multilingual settings often prefer subword/byte fallbacks

Tokenization pitfalls

Whitespace and Unicode normalization

- Whitespace is often meaningful (leading spaces, newlines, tabs)
- Different Unicode forms can look identical but tokenize differently (NFC/NFKC)
- “Smart quotes”, non-breaking spaces, and zero-width characters can change tokens

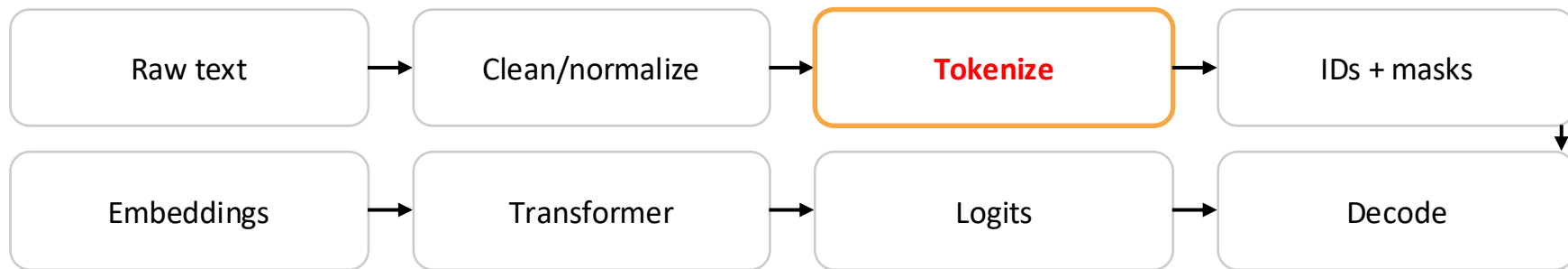
Tokenization pitfalls

Numbers, identifiers, code, and multilingual inputs

- IDs/URLs can become many tokens (e.g., UUIDs, hashes, long paths)
- Code: indentation + symbols matter; token boundaries differ from “words”
- CJK and other scripts may tokenize into shorter pieces; token counts differ per language

End-to-end NLP / LLM pipeline

Where tokenization fits



- Training: next-token prediction (teacher forcing)
- Inference: autoregressive loop with caching (KV cache)
- Tokenization/vocabulary define the model's input and output

Next steps

- Pre Transformer and Transformer (attention)

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?